Report Homework Assignment 2

Mursal Stella Mohammadi

October 9, 2026

1 Introduction

This assignemnt addresses the parallelization of image filtering. Two convolution based filters are applied to an image. Emboss and Sobel. Both operate on a grayscale version of the image (grayscale = 0.3R + 0.59G + 0.11B). The Sobel filter computes the image gradient using two 3×3 kernels, Gx and Gy, resulting in a final pixel value G = sqrt($Gx^2 + Gy^2$). The Emboss filter is based on one kernel and the convolusion formular given in [1]. Both filters shall run in the same execution and produce their output from the same original image. The goal is to implement the filters in two forms. First a sequential program as a baseline and performance reference, then a parallel program using MPI and OpenMP model. MPI distributes image tiles across processes, while OpenMP parallelizes the per-pixel computation within each process. Images are read and written in BMP format using the provided library. Performance needs to be evaluated through time measurements, studying how the solution scales with the number of processes and the size of the input, and quantifying the impact of the number of OpenMP threads.

2 Sequential solution

2.1 Sobel Filter

The program reads a BMP image, applies the Sobel filter, and writes the result to sobel.bmp. SO the process loads the image, filters it and saves it. I implemented the process with the three functions. ImageToGrayscale, GetPixel, and ApplySobel. ApplySobel carries out the filtering operation by first converting the image to grayscale and then convolving it with the two Sobel kernels.

```
...
    // Next two 3 by 3 matrices (or kernels) should be
    //convoluted with the grayscale version of the image A
    // Sobel Matrix Kernel
    const int Gx[3][3] = {
        {-1, 0, 1},
        {-2, 0, 2},
        {-1, 0, 1}};

    const int Gy[3][3] = {
        {1, 2, 1},
        {0, 0, 0},
        {-1, -2, -1}};
...
```

For each pixel at (x, y), I read the 3×3 neighbourhood and gather the weighted sums pixel_x and pixel_y. And I use the given GetPixel function to handle border pixels. The final value is the gradient magnitude:

```
...
// temp buffer for calc
    RGB *temp = (RGB *)malloc(sizeof(RGB) * width * height);
    // convolusion over each pixel
    for (int x = 0; x < width; x++)
    {
        for (int y = 0; y < height; y++)
        {
            // get pixel
```

```
            float pixel_x = 0.0f;
            float pixel_y = 0.0f;

            // Convolusion iterating over neighbouring pixels
            for (int con_x = -1; con_x <= 1; con_x++)
            {
                for (int con_y = -1; con_y <= 1; con_y++)
                {
                    // Get the pixels out of the 3x3 neighbourhood
                    RGB pixel = GetPixel(img, width, height, x + con_x, y + con_y);
                    // +1 to get the correct index in the Gx or Gy matrix
                    // Calc to receive A * Gx or A * Gy
                    pixel_x += pixel.red * Gx[con_y + 1][con_x + 1];
                    pixel_y += pixel.red * Gy[con_y + 1][con_x + 1];
                }
            }
            // G = sqrt(Gx^2 + Gy^2)
            float res = sqrtf(pixel_x * pixel_x + pixel_y * pixel_y);

            // bring value into correct range
            unsigned char final_res = (unsigned char)MIN(MAX(res, 0.0f), 255.0f);
...
```

I bring this value to the range [0, 255] and write it to all three channels of the output pixel.

```
...
// bring value into correct range
            unsigned char final_res = (unsigned char)MIN(MAX(res, 0.0f), 255.0f);

            // get correct index of array
            int index = x + y * width;
            temp[index].red = final_res;
            temp[index].green = final_res;
            temp[index].blue = final_res;
...
```

2.2 Emboss Filter

For the Emboss filter I use a single 3×3 convolution kernel.

```
Kernel = [-2 -1  0]
         [-1  1  1]
         [ 0  1  2]
```

Unlike Sobel, Emboss does not calculate sqrt and no pairing of two kernels. The convolution result is based on the following. [1]

$$g_{x,y} = w * f_{x,y} = \sum_{i=-a}^{a} \sum_{j=-b}^{b} w_{i,j} f_x - i, y - j$$

Because the Emboss kernel has both positive and negative coefficients, the convolution sum can be negative. That is why I use a signed int. And I bound the result afterwards between [0, 255].

```
int sum = 0;
for (int con_y = -1; con_y <= 1; con_y++) {
    for (int con_x = -1; con_x <= 1; con_x++) {
        RGB pixel = GetPixel(img, width, height, x + con_x, y + con_y);
        sum += pixel.red * Kernel[con_y + 1][con_x + 1];
    }
}
unsigned char res = (unsigned char)MIN(MAX(sum, 0), 255);
```

I measure execution time around the call to ApplySobel and ApplyEmboss. The measured interval includes both grayscale conversion and convolutions. And I run the program by passing the input BMP path as a command line argument inside the serial folder.

```
./main bild.bmp
```

It writes the filtered image to sobel.bmp and emboss.bmp in the current directory and prints the elapsed time. If no argument is provided, the program exits and gives an error msg with usage.

3 Parallel solutions

3.1 Parallel

3.1.1 Sobel Filter

For the parallel version I used a hybrid MPI + OpenMP model. MPI distributes the image over processes, and OpenMP parallelizes the per pixel calculation in each process. My goal is to have one MPI rank per cluster node, with each rank using a configurable number of OpenMP threads. I divide the image into horizontal tiles of roughly equal height, one per rank, and let each rank compute the Sobel filter on its tile. Because the 3×3 convolution needs neighbouring pixels, each rank receives its own rows plus one extra row above and below (when they exist) (a "halo"). Only the true rows (halo free) are written back, so no problem when gathering the tiles.

I compute the tiling in GetTiles(), which for a given rank returns:

- start: the first true row owned by this rank
- rows: the number of true rows
- upper_halo and lower_halo: booleans indicating whether an extra row above/below must be added

The last rank takes whatever remains if the division is uneven. Border ranks have only one halo row and middle ranks have two.

BuildScatterGather() then uses GetTiles() for every rank to fill four arrays used by MPI_Scatterv() and MPI_Gatherv().

- total_rows[rank] and start_rows[rank]: counts and offsets including halo rows (used to scatter)
- true_rows[rank] and true_start[rank]: counts and offsets excluding halo rows (used to gather)

This separation ensures that halo data is sent to each rank but not part of the gathering of the final image.

Since the image is stored in rows, I defined a single MPI type, **rowtype**, representing one full row of pixels.

```
MPI_Type_contiguous(width * sizeof(RGB), MPI_BYTE, &rowtype);
MPI_Type_commit(&rowtype);
```

Rank 0 holds the entire image. It scatters each rank's tile (including halo rows) with.

```
MPI_Scatterv(img, total_rows, start_rows, rowtype, tile_img, used_rows, rowtype, 0, MPI_COMM_WORLD);
```

After filtering, each rank gathers its halo free output back into img on rank 0.

```
MPI_Gatherv(tile_output, rows, rowtype, img, true_rows, true_start, rowtype, 0, MPI_COMM_WORLD);
```

I also parallelize the grayscale conversion with a omp parallel for over the tile's pixels.

```
#pragma omp parallel for schedule(static)
for (long i = 0; i < total_pixels; i++) { ... }
```

As the convolution reads the neighbors total_pixels includes the halo rows, so the halo pixels are also converted. And schedule(static) is okay here because each iteration does identical work. **ApplySobelParallel()** uses *collapse(2)* to fuse the row and column loops into a single parallel loop. Also iterating over y (as I am iterating over rows) will be better.

```
#pragma omp parallel
{
    #pragma omp master
    printf("Rank %d: %d threads working\n", rank, omp_get_num_threads());

    #pragma omp parallel for collapse(2) schedule(static)
    for (int y = upper_halo; y < upper_halo + rows; y++)
        for (int x = 0; x < width; x++) { ... }
}
```

The loop starts at y = upper_halo and ends at upper_halo + rows, so only true rows are written. The output index is (y - upper_halo) * width + x, mapping the local row back to the halo free output. And the #pragma omp master block prints how many threads are active per rank. I measure the time around the two parallelized functions and the reported time is the slowest rank's time.

```
#pragma omp parallel
{
  ImageToGrayscaleParallel(...);
  ApplySobelParallel(...);
}
```

For deployment on the IFI cluster I use two helper scripts. refresh_hostfile.sh writes reachable hosts to hostfile and filter_hosts.sh hostfile ¡cores¿ reads hostfile, ssh queries each node's lscpu, and selects only nodes with exactly ¡cores¿ cores. This gives a cluster where one rank per node can use all cores as OpenMP threads. I run the parallel version with:

```
make cluster N=<nprocs> T=<threads> HOSTFILE=<filtered_hostfile> IMG=<image>
```

which runs:

```
mpirun -np N --hostfile HOSTFILE --map-by ppr:1:node --bind-to none \
       --report-bindings -x OMP_NUM_THREADS=T ./main IMG
```

3.1.2 Emboss Filter

The parallel Emboss filter reuses the full parallel structure described for the Sobel Filter. Same tiling (GetTiles), scatter/gather tables (BuildScatterGather), rowtype datatype, MPI_Scatterv and the OpenMP threading. The only additions are the ApplyEmbossParallel() function and two MPI_Gatherv() for each Sobel and Emboss. ApplyEmbossParallel() differs from Sobel Filter in its arithmetic like described in the serial version. The assignment requires both filters to run in the same execution and to produce output from the same original image. For that I have added/changed the Sobel described code.

- Two separate output buffers per rank (*tile_output_sobel and tile_output_sobel*
- Two gathers. After filtering, rank 0 receives the Sobel tile with *MPI_Gatherv(... tile_output_sobel ... sobel_full ...)* and the Emboss tile with MPI_Gatherv(... tile_output_emboss ... emboss_full ...). Two full size receive buffers (sobel_full, emboss_full) are allocated on rank 0 only.
- Two outputs: sobelparallel.bmp and embossparallel.bmp

The parallel program is invoked exactly as explained in the sobel parallel filter version

```
make cluster N=<nprocs> HOSTFILE=<hostfile> T=<threads> IMG=<image>
```

3.2 Optimization

3.2.1 Sobel Filter

In the parallel version each pixel call GetPixel nine times, and each call performs a bounds check

```
(x < 0 || y < 0 || x >= width || y >= height)
```

plus a MIN(MAX(...)) when out of bounds. To that the loop multiplies each neighbour by the kernel coefficient. This includes the zero entries in Gx and Gy, which is redundant. This is why my optimization targets the inner convolution kernel, leaving the MPI decomposition and the OpenMP parallel structure unchanged. Both versions distribute the same tiles and use the same thread model.

I rewrote ApplySobelParallel() so that each row is handled with three pre computed row pointers and no per pixel GetPixel calls.

```
...
RGB *up   = tile_img + (size_t)MAX(y - 1, 0) * width;
RGB *mid  = tile_img + (size_t)y * width;
RGB *down = tile_img + (size_t)MIN(y + 1, local_height - 1) * width;
RGB *out        = tile_output + (size_t)(y - upper_halo) * width;
...
```

The (MAX(y-1, 0) and MIN(y+1, local_height-1)) is computed once per row instead per neighbour lookup. This alone removes 9 bounds checks per pixel, replacing them with 2 integer comparisons per row. The per pixel calculation is moved into a helper function, SobelPixel(), which receives the left, middle, and right column indices. The middle column of Gx and the middle row of Gy are zeros. Multiplied by zero can be dropped, reducing convolution from 9 to 6 terms. The remaining coefficients are ±1 and ±2. Instead of writing the general form A * K, I factor out the sign and write the sum as a difference of two sums.

```
void SobelPixel(const RGB *up, const RGB *mid, const RGB *down,
                int xl, int x, int xr, RGB *out, int idx)
{
    int gx = (up[xr].red + 2 * mid[xr].red + down[xr].red)
           - (up[xl].red + 2 * mid[xl].red + down[xl].red);
    int gy = (up[xl].red + 2 * up[x].red + up[xr].red)
           - (down[xl].red + 2 * down[x].red + down[xr].red);

    float res = sqrtf((float)(gx * gx + gy * gy));
    unsigned char final_res = (unsigned char)MIN(res, 255.0f);
    out[idx].red = out[idx].green = out[idx].blue = final_res;
}
```

The column loop is then split into three parts in ApplySobelParallel(). The left and right edge pixels are handled once and the inner loop has no bounds checks at all.

```
...
SobelPixel(up, mid, down, 0, 0, MIN(1, width - 1), out, 0);
for (int x = 1; x < width - 1; x++)
    SobelPixel(up, mid, down, x - 1, x, x + 1, out, x);
SobelPixel(up, mid, down, MAX(width - 2, 0), width - 1, width - 1, out, width - 1);
...
```

The Sobel kernels are

```
...
Gx = [-1  0  1]        Gy = [ 1  2  1]
     [-2  0  2]             [ 0  0  0]
     [-1  0  1]             [-1 -2 -1]
...
```

I also split the time measurement into two phases. Compute phase: grayscale + Sobel, measured with MPI_Barrier before the timer and reduced with MPI_MAX. And the Assembly phase: the MPI_Gatherv call, measured the same way. Printing both lets me distinguish improvements in the calculation and the gathering phase as this depends on the number of the processes.

The optimized code can be for example be executed like the following through the Makefile.

```
make opt N=10 HOSTFILE=hostfile_8 T=8 IMG=large.bmp
```

This builds opt_main from opt.c and launches it with mpirun -np 10 on the nodes listed in hostfile_8, mapping one rank per node (–map-by ppr:1:node), disabling CPU binding so OpenMP threads can use all cores (–bind-to none), and setting OMP_NUM_THREADS=8 for each rank.

MPI Ranks (p)	`mainMPI` **Time (s)**	`optMPI` **Time (s)**	**Performance Gain**
1	172.04	171.31	+0.42%
4	42.77	42.89	-0.28% (Noise)
8	21.48	21.47	+0.05%
12	14.41	14.33	+0.56%
16	10.78	10.81	-0.28% (Noise)
20	10.19	8.62	**+15.38%**
24	9.65	7.20	**+25.39%**

Table 1: Comparative performance between `mainMPI` and `optMPI` ($L = 4$).

3.2.2 Emboss Filter

I applied the same optimization to the Emboss filter as described for the Sobel. Here we have two zero coefficients resulting in 7 instead of 9 coefficients to be calculated. The optimized version executed the same way as in the parallel version.

```
make opt N=<processes> HOSTFILE=<hostfile> T=<threads> IMG=<image>
```

4 Performance evaluation

Hardware and System Environment

All measurements were carried out on the IFI cluster. I used refresh_hostfile.sh to collect available nodes and filter_hostfile.sh to select only those with 8 cores per node and 1 thread per core, for benchmarking. I then used gather.sh and summarize.sh to query each node's lscpu and extract only the relevant fields. Out of 25 nodes, 24 share the same configuration, and 1 differs.

```
—— 24 node(s) ——
Model name: Intel(R) Core(TM) i7-9700 CPU @ 3.00GHz
Thread(s) per core: 1
Core(s) per socket: 8
Socket(s): 1
NUMA node(s): 1

—— 1 node(s) ——
Model name: 11th Gen Intel(R) Core(TM) i9-11900KF @ 3.50GHz
Thread(s) per core: 1
Core(s) per socket: 8
Socket(s): 1
NUMA node(s): 1
```

Performance Metrics

I use three metrics:

Metric	Formula	Description
Execution Time (T_p)	`clock_gettime`	Elapsed time in seconds (the slowest rank).
Speedup (S_p)	$S_p = \frac{T_1}{T_p}$	Single-process time divided by p-process time.
Efficiency (E_p)	$E_p = \frac{S_p}{p}$	How close to ideal linear speedup ($0 \leq E_p \leq 1$).

Table 2: Definitions of the parallel scaling metrics.

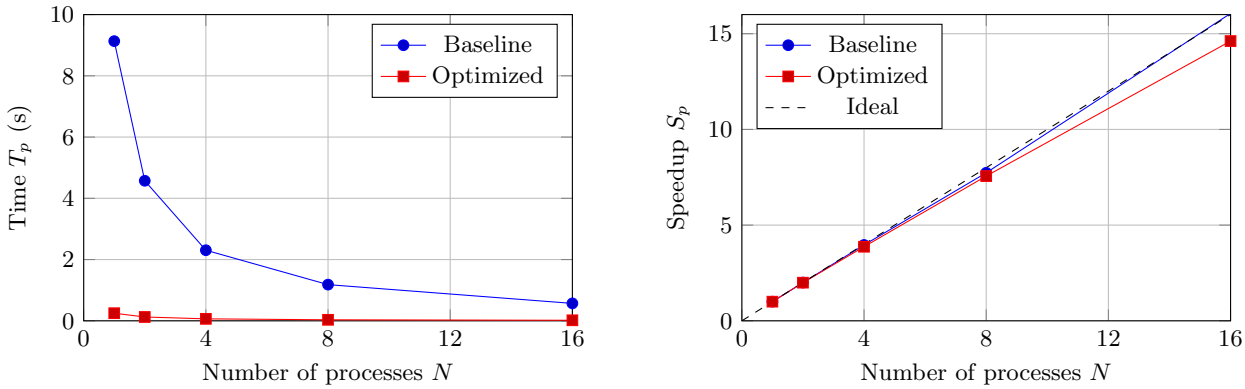
Strong Scaling Analysis

The image size is fixed to the largest image **wallpaper_8** with 983 MB (1.030.874.716 Bytes), while the number of MPI processes N varies. Each process runs on its own node, and the number of OpenMP threads per process is fixed at $T = 8$, matching the core count of the benchmark nodes. I use the parallel code at $N = 1$ with the same T as the reference time T_1. The serial time (17.127 s) is reported separately as a reference. For the baseline parallel version $T_1 = 9.133$ s, and for the optimized version $T_1 = 0.248$ s. The following table lists the measured times for both the baseline parallel code and the optimized code, together with the speedup $S_p = T_1/T_p$ and efficiency $E_p = S_p/N$.

The baseline parallel version scales almost perfectly up to $N = 8$, with efficiency remaining very high. At $N = 16$ the measured efficiency slightly exceeds 1.0. The optimized version shows the same near-linear scaling but with a small systematic drop at higher process counts ($E_p = 0.913$ at $N = 16$).

	Baseline parallel ($T = 8$)			**Optimized** ($T = 8$)		
N	T_p (s)	S_p	E_p	T_p (s)	S_p	E_p
1	9.1328	1.000	100%	0.2481	1.000	100%
2	4.5729	1.997	99.9%	0.1245	1.992	99.6%
4	2.3050	3.962	99.1%	0.0641	3.870	96.8%
8	1.1811	7.733	96.7%	0.0328	7.562	94.5%
16	0.5687	16.059	100.4%	0.0170	14.616	91.3%

Table 3: Strong scaling measurements for the largest image, $T = 8$ threads per rank, one rank per node. Speedup and efficiency are computed against the corresponding $N = 1$ time at the same T.



(a) Execution time.

(b) Speedup.

Figure 1: Strong scaling on wallpaper_8 with $T = 8$. Left: time. Right: speedup. Linear axes.

Input Size Scaling

Next is how execution time scales with input size, keeping the number of MPI processes and the number of OpenMP threads fixed. I use $N = 20$ processes and $T = 8$ threads per process. The base image is wallpaper.bmp (23.7 MB, 24,883,254 bytes), and I scale it by factors of 1×, 2×, 4×, and 8× by combining copies vertically with ImageMagick as suggested in the assignment. The image content is therefore repeated, but the number of pixels processed grows exactly with the stated factor.

The table lists the measured times for both the parallel and optimized versions.

Size	Baseline T_p (s)	Opt. T_p (s)	Speedup
1×	0.0931	0.00224	41.6
2×	0.1164	0.00430	27.1
4×	0.2285	0.00757	30.2
8×	0.4601	0.01420	32.4

Table 4: Execution time versus image size factor ($N = 20$, $T = 8$).

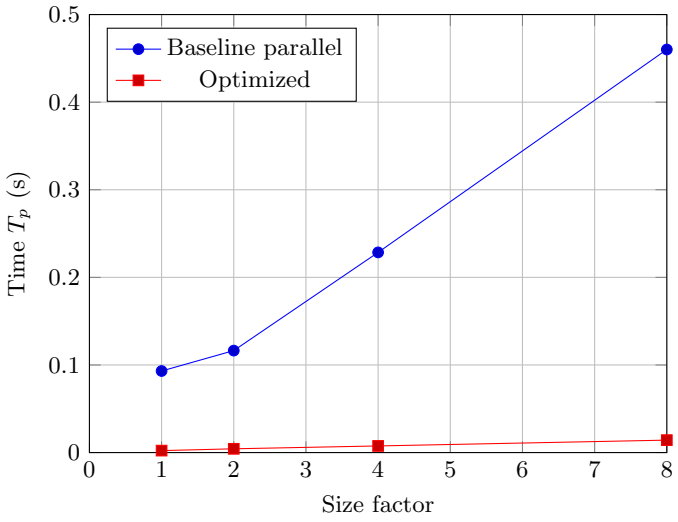


Figure 2: Execution time versus image size factor.

Figure 3: Image size scaling ($N = 20$, $T = 8$).

Weak Scaling Analysis

I keep the work per process constant and grow the problem size in proportion to the number of processes. Starting from $N = 1$ with the base image (wallpaper.bmp) (1×), I use 2× at $N = 2$, 4× at $N = 4$, and 8× at $N = 8$. I use $T = 8$ threads per process throughout. Both versions have pretty good weak scaling. The

	Baseline parallel ($T = 8$)			**Optimized** ($T = 8$)		
N	Size	T_p (s)	E_{weak}	Size	T_p (s)	E_{weak}
1	1×	1.1123	100%	1×	0.03279	100%
2	2×	1.1624	95.7%	2×	0.03272	100.2%
4	4×	1.1236	99%	4×	0.03273	100.2%
8	8×	1.1353	98%	8×	0.03275	100.1%

execution time is essentially constant from $N = 1$ to $N = 8$ despite the problem growing eightfold.

Thread Scaling

Last, I scale the number of OpenMP threads per process while keeping the image size and the number of MPI processes fixed. I use the 8× wallpaper.bmp and $N = 2$ processes. Thread counts are $T \in \{1, 2, 4, 8\}$, which goes up to but not beyond the physical core count of the benchmark nodes, so no oversubscription occurs. The following table lists execution times together with thread speedup and efficiency, defined as $S_T = T_1/T_T$ and $E_T = S_T/T$.

	Baseline parallel ($N = 2$)			**Optimized** ($N = 2$)		
T	T_p (s)	S_T	E_T	T_p (s)	S_T	E_T
1	4.4477	1.000	100%	0.9479	1.000	100%
2	3.4695	1.282	64.1%	0.4739	2.000	100%
4	2.9650	1.500	37.5%	0.2464	3.846	96.2%
8	2.8071	1.584	19.8%	0.1245	7.614	95.2%

The baseline parallel version scales poorly with thread count. doubling the threads from 1 to 2 gives only

a 1.28× speedup, and going from 4 to 8 threads reduces the time by less than 6%. Efficiency drops from 1.0 at $T = 1$ to 0.20 at $T = 8$. The optimized version, in contrast, scales nicely. $T = 2$ gives exactly 2.0× speedup, and even at $T = 8$ the efficiency remains above 0.95.

5 Discussion

My solution scales pretty well across almost all evaluations. Along side I have also measured the time MPI_Gatherv() needs to assemble the image tiles. And with N growing the time also grows, making runtime grow as well. So this could be part of future improvement.

6 Conclusion

I implemented a hybrid MPI + OpenMP solution for Sobel and Emboss filtering. The image is split into row tiles with halo rows included in the scatter and OpenMP parallelizes the per-pixel work within each rank. The results confirm good scaling. strong scaling stays above 0.91 efficiency up to $N = 16$, weak scaling is essentially flat, and optimized thread scaling reaches 0.95 efficiency at $T = 8$. The main optimization was replacing per-pixel `GetPixel` calls with row pointers and arithmetic simplification.

References

[1] Wikipedia contributors: Kernel (image processing). Wikipedia, The Free Encyclopedia. `https://en.wikipedia.org/wiki/Kernel_(image_processing)`